Ajayi OluwaPelumi David

Implementing and Evaluating a Simple Information Retrieval System over the Project Gutenberg Collection

Abstract

This project describes the implementation and evaluation of a simple Information Retrieval (IR) system built over a local copy of the Project Gutenberg collection. The system indexes a large collection of plain-text books and supports query-based retrieval using three different approaches: structured metadata search, a Vector Space Model based on TF-IDF weighting, and the BM25 ranking function. All indexing and retrieval mechanisms were implemented from scratch in Python using standard libraries, in accordance with the assignment requirement to avoid external search libraries. The system was evaluated using six predefined queries representing different types of search problems, including phrase queries, topical queries, author names, rare terms, and multilingual text. The evaluation shows that structured search performs well for metadata queries, while BM25 provides the most balanced and robust performance for full-text retrieval across the collection.

1. Introduction

Information Retrieval (IR) systems are designed to retrieve relevant documents from large collections in response to user queries. Search engines rely on efficient indexing methods and ranking algorithms to estimate how relevant documents are to a user's query. The goal of this project was to implement a simple search engine capable of indexing a collection of books from the Project Gutenberg dataset and retrieving relevant books based on textual queries. The assignment required implementing three different retrieval models: structured metadata search, a Vector Space Model using TF-IDF, and the BM25 ranking function. Unlike many modern search systems that rely on external frameworks such as Lucene or Elasticsearch, this project required implementing all indexing and ranking mechanisms manually. The system was therefore built entirely using Python and standard libraries. The implemented system supports indexing thousands of books, processing textual queries, ranking documents according to different scoring models, and generating result files containing ranked document lists.

System Architecture

The implemented search engine follows a standard Information Retrieval pipeline consisting of multiple stages:

Text Collection → Preprocessing → Inverted Index Construction → Ranking → Result Output

The dataset used in this project consists of:

a. Plain-text versions of books from Project Gutenberg.
b. A metadata CSV file containing structured information such as title, author, author id, bookshelf category, rights and language.

The architecture of the system consists of four main components.

1. Text Preprocessing
2. Inverted Index Construction
3. Query Processing and Ranking
4. Result Generation and Evaluation

Each component plays a crucial role in ensuring efficient retrieval and meaningful ranking of documents.

2. Indexing

2.1 Text Preprocessing

Before indexing, the raw text files must be cleaned and normalized to ensure consistent representation of terms. Each document in the dataset undergoes several preprocessing steps.

First, Project Gutenberg boilerplate text is removed. Gutenberg books typically contain introductory licensing text and metadata that appear before and after the main content. These sections were removed by detecting the markers indicating the start and end of the actual book content. Next, the remaining text is converted to a lowercase. This ensures that terms such as “Book” and “book” are treated as the same token.

The text is then tokenized using regular expressions. Tokenization splits the document text into individual terms by removing punctuation and separating words. Special attention was given to preserving German characters such as ä, ö, ü, and ß. This allows the system to correctly handle multilingual queries such as “Dornröschen”.

Stop word removal and stemming were intentionally not applied in this implementation. While these techniques can improve retrieval quality, the goal of this project was to observe the behavior of ranking models under basic preprocessing conditions.

2.2 Inverted Index Construction

To support efficient search, an inverted index was constructed from the preprocessed documents.

The inverted index stores a mapping from each term to the documents that contain it:

term → {document_id : term_frequency}

In addition to the inverted index itself, several statistics were computed and stored during indexing:

a. Document length (number of tokens in each document).
b. Document frequency (number of documents containing a term).
c. Total number of indexed documents.
d. Average document length across the collection.

These statistics are required for computing TF-IDF weights and BM25 scores during retrieval.

The inverted index allows the system to retrieve only the documents that contain query terms instead of scanning the entire collection, which significantly improves search efficiency.

3. Ranking

The system implements three different retrieval models that rank documents according to different relevance criteria.

3.1 Structured Metadata Search

Structured search operates on metadata rather than the full document text. The metadata file contains structured fields including title, author, bookshelf category, and language.

During retrieval, each query term is matched against these metadata fields. A weighted scoring scheme is used to reflect the importance of different fields:

a. Title match: +3.0
b. Author match: +2.0
c. Bookshelf match: +1.0
d. Language match: +0.5

The final score of a document is calculated as the sum of these weights across all query terms.

Structured search is particularly effective for queries targeting specific authors or book titles, but it cannot capture relevance based on the full text of documents.

3.2 Vector Space Model (TF-IDF)

The Vector Space Model represents documents and queries as vectors in a high-dimensional term space.

Each term receives a weight based on its frequency in the document and its rarity across the collection. The inverse document frequency is calculated using the following formula:

$$idf(t) = \log\left(\frac{N+1}{df(t)+1}\right) + 1$$

where *NN*N is the number of documents and *df(t)df(t)*df(t) is the document frequency of term *tt*t.

The TF-IDF weight for a term in a document is calculated as:

$$w_{t,d} = tf_{t,d} \cdot idf(t)$$

Document similarity to the query is then computed using cosine similarity:

$$sim(q,d) = \frac{q.d}{||q||.||d||}$$

This approach helps rank documents that contain important query terms higher while also accounting for document length.

3.3 BM25 Ranking

BM25 is a probabilistic ranking function that improves upon TF-IDF by incorporating document length normalization and saturation of term frequency.

The BM25 score for a term is calculated using:

$$score(t,d) = idf(t) \cdot \frac{tf(t,d)(k_1+1)}{tf(t,d) + k_1\left(1 - b + b\frac{dl}{avgdl}\right)}$$

where:

a. tf(t,d) is the term frequency in the document
b. dl is the document length
c. *avgdl* is the average document length
d. k1 and b are tuning parameters

In this implementation the parameters were set to:

a. k_1 = 1.5
b. B = 0.75

BM25 provides better normalization for long documents and generally produces more stable rankings.

4. Evaluation

The system was evaluated using six predefined queries representing different search scenarios.

Query 1: "to be, or not to be"

This query contains very common English words. Because stop words were not removed during preprocessing, these terms appear in many documents across the collection. As a result, many documents receive similar scores.

This query demonstrates how frequent terms reduce ranking discrimination. Structured search performs poorly because such phrases rarely appear in metadata fields.

Query 2: "English Grammar"

This query represents a topical search query. Structured search performs well when the terms appear directly in book titles related to grammar or language instruction.

The TF-IDF and BM25 models also retrieve relevant educational texts where the terms appear within the document content. Since the terms are moderately specific, IDF weighting provides reasonable discrimination between relevant and non-relevant documents.

Query 3: "Philip K Dick"

This query shows one of the limitations of simple bag-of-words retrieval models.

Structured search returns documents whose metadata contains substrings such as "Dickens" or "Dickinson", because substring matching was used for metadata fields. These results are not necessarily related to the intended author. The BM25 model retrieves documents containing the terms "Philip" and "Dick" independently within the text, which may refer to different individuals.

This query demonstrates that simple term-based retrieval models cannot capture the semantic meaning of a complete name as a single entity.

Query 4: "Jabberwocky"

"Jabberwocky" is a rare term within the dataset. Because its document frequency is low, the inverse document frequency value is high.

As a result, the BM25 model strongly favors documents containing the term and ranks relevant books discussing Lewis Carroll's poem at the top of the results.

This query demonstrates the effectiveness of IDF weighting for rare terms.

Query 5: "Gutenberg"

The term "Gutenberg" appears frequently in the dataset, particularly in metadata and introductory text sections. Even after removing boilerplate content, the term remains common across many documents.

This leads to reduced discrimination between documents and illustrates how dataset bias can affect ranking performance.

Query 6: "Dornröschen"

This query tests the system's ability to handle multilingual text. Because the preprocessing step preserved German characters such as ö, documents containing "Dornröschen" were correctly indexed and retrieved.

This confirms that the tokenization process supports non-English terms.

Model	Strengths	Weakness
Structured Search	Effective for media queries	Ignores full document text
TF-IDF	Simple and Intuitive ranking	Limited length normalization
BM25	Robust ranking and good length normalization	Slightly more complex

Overall, BM25 produced the most consistent results across different query types.

Comparison Across Retrieval Models

To compare the performance of the three retrieval approaches, the ranked results produced by each model were examined for the six evaluation queries. Structured search performed best for queries that directly match metadata fields, such as author names or titles, since it explicitly searches fields like title and author. However, it cannot retrieve documents based on their full textual content. The TF-IDF Vector Space Model was able to retrieve documents containing relevant terms within the document text, but it's ranking sometimes favored longer documents because it does not normalize document length as effectively as BM25. The BM25 model generally produces the most balanced rankings across queries, particularly for rare or informative terms such as "Jabberwocky". This suggests that BM25's length normalization and probabilistic scoring provide more stable results across different query types.

5. Conclusion

This project implemented a simple search engine capable of indexing and retrieving documents from the Project Gutenberg collection. Three retrieval models were implemented from scratch: structured metadata search, a TF-IDF Vector Space Model, and BM25. The system was designed to process plain-text books, build an inverted index, and return ranked search results for user queries.

The evaluation showed that the different retrieval models behave differently depending on the type of query. Structured search performed well when the query matched information stored in metadata fields such as author or title. In contrast, the TF-IDF and BM25 models were able to retrieve documents based on the actual content of the books. Among the three approaches, BM25 generally produced the most balanced rankings because it considers both term frequency and document length normalization.

The experiments also highlighted some limitations of basic bag-of-words retrieval models. For example, queries containing very common words resulted in many documents receiving similar scores, which reduces ranking discrimination. Similarly, queries containing names such as "Philip K Dick"

demonstrate that simple term-based models cannot easily distinguish between different entities when words appear independently in documents.

There are several ways the system could be improved in future work. One possible improvement would be to introduce stop word removal or stemming in order to reduce the impact of very frequent terms and normalize different word forms. Phrase queries or proximity-based ranking could also help improve retrieval for queries that contain well-known expressions or names. In addition, techniques such as named entity recognition could help the system better understand queries referring to specific people or works. Another possible improvement would be to optimize the indexing process for larger datasets. While the current implementation stores the index in memory, a disk-based indexing approach would allow the system to scale to even larger collections of documents.

Overall, this project provided practical experience in implementing core Information Retrieval techniques. Building the indexing and ranking mechanisms from scratch helped demonstrate how search engines organize large text collections and how different ranking models influence the quality of retrieval results.

References

[1] C. D. Manning, *Introduction to information retrieval*. Syngress Publishing, 2008.

[2] S. Robertson and H. Zaragoza, *The probabilistic relevance framework: BM25 and beyond*, vol. 4. Now Publishers Inc, 2009.